

Software Project Management

Software Metrics

Outline

- Metrics for Process & Projects”
 - Measurement
 - Measures
 - Metrics
 - Software Metrics
 - Process metrics
 - Project metrics
 - Direct and Indirect software measures
 - Software Measurement
 - Size-oriented Metrics
 - Function-oriented Metrics
 - Object-oriented Metrics
 - Use-case-oriented Metrics
 - Software Quality Metrics
 - Defect Removal Efficiency

Introduction

- *Software process and project metrics*
 - are quantitative measures
 - enable software engineers to gain insight into efficacy of the software process and projects
- Basic quality & productivity data (called measures) are
 - Collected (and normalized using size- or function-oriented metrics),
 - analyzed,
 - compared against past averages,
 - and assessed to determine whether productivity & quality improvements have occurred
- Metrics also indicate problem areas in processes and projects

What is Measurement?

- **It is the act of obtaining a measure**
- It is a mechanism of objective evaluation
 - enables managers and practitioners to improve software process
 - assists in planning, tracking and controlling a software project and assessing product quality

What is a Measure?

- It provides a quantitative indication of the size of some product or process attribute

OR

- quantitative indication of the extent, amount, dimension, capacity, or size of some attribute of product or process

Metric?

- **A metric is quantitative measure of the degree to which a system, component or process possesses a given attribute (IEEE definition)**
- Metrics enable people to gain insight into the effectiveness of the software process and the projects

Difference between Measure, Metric, Indicator

■ Measure

- It is a value which does not provide enough information to make meaningful decisions

■ Metric

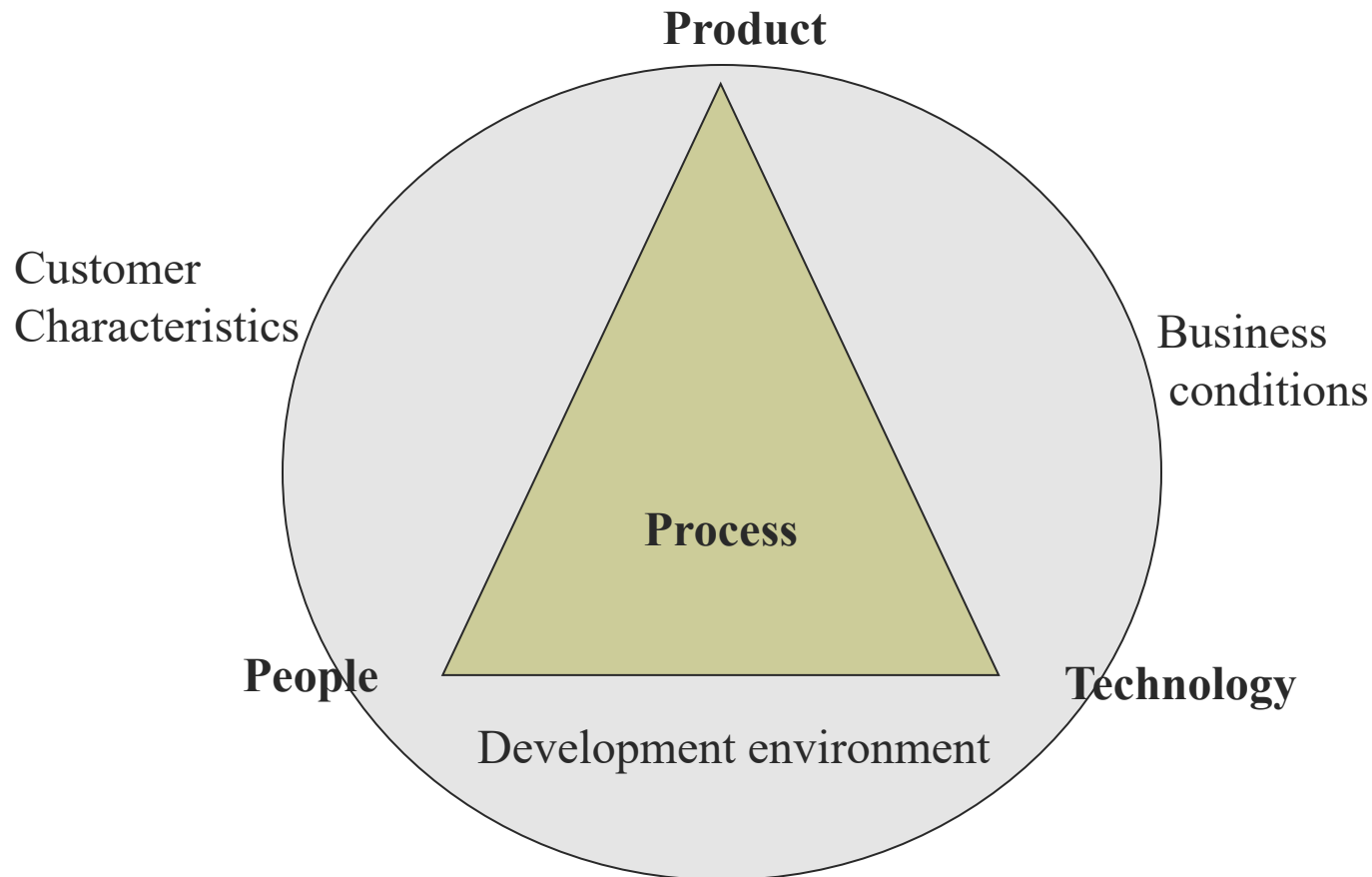
- A comparison of two or more measures, e.g., temperature values over time, errors per KLOC

■ Indicator

- Compares a metric against a baseline or expected result and helps in making decisions

Determinants for software quality and organizational effectiveness

Process connects 3 important factors that influence software quality and organizational performance



Process Metrics & Software Process Improvement

- To improve a process
 - Measure specific attributes of process
 - Develop meaningful metrics based on these attributes
 - These metrics provide indicators that lead to strategy for improvement
- Example
 - Metrics can be derived from the following process outcomes/attributes
 - Errors uncovered before software release,
 - defects delivered to & reported by end-user,
 - work products delivered
 - Human efforts expended
 - Calendar time expended
 - Schedule conformance

Project Metrics

- Used by project managers and software team to adapt project workflow and technical activities
- Objective is
 - To minimize the development schedule by making adjustments that can avoid delays and mitigate potential problems
 - To assess product quality on an ongoing basis and when necessary, modify technical approach to improve quality
 - Most important use of metrics is estimation of time, effort.

Project Metrics

■ Example

- Metrics collected from past projects help to establish time and effort estimates for current software projects
- Production rates (in terms of models created, review hours, function points, delivered source lines)
- Errors uncovered during each software engg. Task

■ Software projects are assessed against the project metrics for improvement

Software Measurement Categories

■ Direct Measures

- Of *software process* are
 - Cost and effort
- Of *product* are
 - Line of Code (LOC)
 - Execution Speed
 - Defects over specified period of time

■ Indirect Measures

- Of *product* are
 - Functionality
 - Quality
 - Complexity
 - Efficiency
 - Reliability
 - Maintainability
 - .. Other Abilities

Size Oriented Metrics

- Derived by normalizing quality and/or productivity measures by considering the *size* of software that has been produced.
- LOC used as a normalizing factor for other measures
- Widely used in software industry but their validity and applicability is debated

Example Set of Size-oriented metrics

- Errors per KLOC (thousand lines of code)
- Defects per KLOC
- Cost per LOC
- Pages of Documentation per KLOC
- Errors per person-month
- LOC per person-month
- Cost per Page of documentation

Pros and cons

- Though widely used but not universally accepted as best way to measure a software process
- Controversy lies in using LOC as a key measure
- proponents claim
 - LOC easy to count
 - Many existing estimation models use LOC
 - Large literature & data based on LOC exists
- But opponents argue that
 - LOC measures are prog. language dependent
 - When considering productivity, LOC criteria penalizes well designed short programs
 - Can not accommodate non procedural languages
 - Planner must estimate LOC long before analysis and design

Function-oriented Metrics

- These use a measure of the functionality delivered by application as a normalization value.
- Most widely used function-oriented metric is function point (FP)
- FP's computation – based on characteristics of software information domain and complexity.

Function Point

- First proposed by Albrecht; can be used to measure functionality delivered by a system.
- Functional Point Analysis (FPA) is a software measurement technique used to assess the size and complexity of a software system based on its functionality.
- It involves categorizing the functions of the software, such as input screens, output reports, inquiries, files, and interfaces, and assigning weights to each based on their complexity.
- By quantifying these functions and their associated weights, FPA provides an objective measure of the software's size and complexity.

Computing function points (cont..)

- Following empirical relationship is used to compute Function Point (FP)

$$FP = \text{total count} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

F_i ($i=1$ to 14) are VAF (Value adjustment factors) or simply 'adjustment values' based on answers to some questions.

An Example of FP-Based Estimation

- Decomposition for FP-based estimation focuses on information domain values rather than software functions. Following estimates can be made
 - estimate inputs
 - outputs,
 - Inquiries
 - Files
 - external interfaces for the CAD software.

The mechanical CAD software will accept two- and three-dimensional geometric data from a designer. The designer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of devices. The software will be designed to control and interact with peripheral devices that include a mouse, scanner, laser printer, and plotter.

Information domain value	Opt.	Likely	Pess.	Est. count	Weight	FP count
Number of external inputs	20	24	30	24	4	97
Number of external outputs	12	15	22	16	5	78
Number of external inquiries	16	22	28	22	5	88
Number of internal logical files	4	4	5	4	10	42
Number of external interface files	2	2	3	2	7	15
<i>Count total</i>						320

Finally, the estimated number of FP is derived:

$$FP_{\text{estimated}} = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)] = 375$$

An Example of LOC-Based Estimation

As an example of LOC and FP problem-based estimation techniques, we consider a software package to be developed for a computer-aided design application for mechanical components. The software is to execute on a desktop workstation and must interface with various computer graphics peripherals including a mouse, digitizer, high-resolution color display, and laser printer. A preliminary statement of software scope can be developed:

The mechanical CAD software will accept two- and three-dimensional geometric data from a designer. The designer will interact and control the CAD system through a user interface that will exhibit characteristics of good human/machine interface design. All geometric data and other supporting information will be maintained in a CAD database. Design analysis modules will be developed to produce the required output, which will be displayed on a variety of devices. The software will be designed to control and interact with peripheral devices that include a mouse, scanner, laser printer, and plotter.

Estimation table for the LOC methods

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
<i>Estimated lines of code</i>	<i>33,200</i>

- Software estimation metrics are essential tools in project management, helping teams predict the time, effort, and cost required to complete a software development project. Some of the commonly used software estimation metrics and techniques are:

- 1. **Lines of Code (LOC):**

- **Description:** Measures the size of a software program by counting the number of lines in the source code.
 - **Use:** Useful for estimating effort and cost in projects where historical data is available.

- 2. **Function Points (FP):**

- **Description:** Measures the functionality delivered to the user, based on the number of inputs, outputs, user interactions, files, and external interfaces.
 - **Use:** Helps in estimating the size and complexity of software, making it easier to compare different projects.

- 3. **Use Case Points (UCP):**

- **Description:** Estimates effort based on use cases, considering factors such as actor complexity, use case complexity, and technical/environmental factors.
 - **Use:** Suitable for projects using use case modeling, often in agile environments.

COCOMO

- Separate PPT is shared.

Object-oriented Metrics

- Used for object-oriented projects
- Set of metrics for OO projects:
 - Number of scenario scripts
 - Scenario scripts are detailed sequence of steps about user and application interaction
 - Scenario scripts are directly correlated to application size and no. of test cases
 - Number of key classes
 - Key classes are independent components
 - They Indicate amount of development effort and potential reuse
 - Number of support classes
 - These indicate amount of development effort and potential reuse
 - Average number of support classes per key class
 - Number of sub systems (aggregation of classes)
 - If identified, it is easier to lay out the schedule

Use-case oriented Metrics

- Use-cases describe user visible functions and features
- They are defined early in software process and can be used as normalization measure before significant activities are initiated
- They are independent of programming language
- No. of use cases directly proportional to
 - size of application in LOC &
 - no. of test cases that will be designed
- There is no standard size of a use case as they are created at different levels of abstraction
 - For this reason, it is a suspect as a normalization measure

Software Quality Metrics

- Measuring quality through
 - Correctness
 - It is degree to which software performs its required function
 - Common measure= defects per KLOC
 - For quality assessment defects are counted typically for 1 year
 - Maintainability
 - It is the ease with which a program can be
 - corrected if an error is found
 - Adapted if environment changes or
 - Enhanced if customer desires
 - Measured indirectly, e.g., Mean-time-to change (MTTC)
 - Maintainable programs -> lower MTTC

Software Quality Metrics

- Integrity

- System's ability to withstand (accidental & intentional) attacks
- Two more attributes that help determine integrity
 - threat = probability of attack within a given time (that causes failure)
 - security = probability that an attack will be repelled
- $\text{Integrity} = \Sigma [1 - (\text{threat} * (1 - \text{security}))]$

- Usability

- It quantifies ease of use and learning (time)

Defect Removal Efficiency

- It is a measure of the filtering ability of the quality assurance and control activities as they are applied through out the process framework.
- $DRE = E / (E + D)$
 - E = # of errors found before delivery
 - D = # of defects found after delivery
- Ideal value of DRE is 1
- Realistically D will be greater than 0
- As E increases, DRE begins to approach 1

Redefining DRE

- DRE can also be applied on each process framework activity and hence find the team's ability to assess errors before they are passed to next activity or software engineering task.
- $DRE = E_i / (E_i + E_{i+1})$
 - E_i = errors in activity i
 - E_{i+1} = errors in activity i+1 that were not discovered in activity i